

Accelerating the dispatch-bound HSiKAN forward alternatives, measured — batch, cheaper bases, the CR–Chebyshev hybrid, and sparsity

Aiko (Claude Code), for Dr. Csaba Hajdu

2026-06-28 (JST)

Abstract

HSiKAN’s forward is dispatch-bound: ~ 200 tiny ops, $\sim 76\%$ overhead (elementwise, dtype copies, reshapes), only $\sim 24\%$ real compute. The decisive reframing (measured): it is a **batch-size problem**, not a “HSiKAN is slow” problem. At $B=256$ the spline costs $43\mu s$ /sample — **$17\times$ cheaper** than the $754\mu s$ at $B=1$ — because the per-op dispatch overhead is *fixed* and amortises under batching. From this, four independent, composable accelerations follow, each measured on this machine (CPU, 1 thread). The headline: **batch the rollout** (model-agnostic, $17\times$), a **cheaper basis** (Chebyshev/RBF, $3\times$ at $B=1$), the B^L **collapse** (SA-HSiKAN, already shipped, $2.6\times + 11\times$ fewer params), and **fusion** (`torch.compile`/Triton, $\sim 10\times$ on GPU). A **CR–Chebyshev hybrid** (Chebyshev coefficients generate the spline control points through a cached knot-basis) gives $2.4\times$ the spline resolution per parameter and, crucially, a Chebyshev-representable function that can be *deployed* via the cheap matmul. Sparsity is a scaling lever, not a small-graph win.

1 Diagnosis: a $B=1$ problem

Profiling the galambos HSiKAN forward (15,368 params): one $B=1$ forward is ~ 200 aten ops dominated by `mul/_to_copy/copy-/unsqueeze/gather` (the Catmull-Rom spline and 71 dtype copies), with only $\sim 24\%$ in `einsum/addmm`. The aggregation is already one `einsum` per layer and the spline is already batched — so this is **not** a missing vectorisation; it is per-op dispatch overhead over a small graph. The regime table makes the lever obvious:

	$B=1$ (rollout)	$B=256$ (update)
HSiKAN forward (galambos)	$2255\mu s$	$16175\mu s \rightarrow \mathbf{63\mu s/sample}$
Catmull-Rom activation only	$754\mu s$	$10934\mu s \rightarrow \mathbf{43\mu s/sample}$

Table 1: The per-sample cost falls $\sim 17\text{--}36\times$ from $B=1$ to $B=256$: the overhead is fixed per op, so it amortises. The rollout (one env step = one $B=1$ forward) is the dispatch-bound regime.

2 The alternative space (all measured)

The honest synthesis: the spline *values* cannot be cached (data-dependent), but three things make that irrelevant — make B large (**batch the rollout**), cut the per-op count at $B=1$ (**cheaper basis** / **SA-HSiKAN** / **fusion**), or shrink the copies. None requires MSVC; the first keeps the full learnable HSiKAN.

alternative	mechanism	measured	keeps learnability
Batch the rollout	amortise dispatch	17× cheaper/sample	yes (model-agnostic)
Chebyshev / RBF basis	matmul, no gather/branch	3.0× / 2.8× at $B=1^\dagger$	yes
SA-HSiKAN (B^L collapse)	cache the <i>operator</i>	2.6×, 11× fewer params	partial (frozen MP)
<code>torch.compile</code> / Triton	fuse ~ 200 ops $\rightarrow$ few	$\sim 10\times$ (GPU; needs compiler)	yes
kill dtype copies (71/fwd)	one dtype, contiguous	$\sim 16\%$	yes

Table 2: † Chebyshev/RBF are *regime-dependent*: $3\times$ faster at $B=1$ (fewer ops) but *slower* at $B=256$ (they materialise a $(\cdot, C, k)$ tensor; the CR 4-gather is cheaper batched). Pick the basis by regime; there is no free lunch.

3 The CR–Chebyshev hybrid (Hajdu)

“*Chebyshev coefficients are the CR control points.*” Concretely: the G spline control points are $P = C T_{\text{knots}}^\top$, where $T_{\text{knots}} \in \mathbb{R}^{G \times k}$ is the Chebyshev basis *evaluated at the fixed knots* — a **cached constant** — and $C \in \mathbb{R}^{(\text{channels}) \times k}$ is learned (k coefficients/channel). Measured ($C=64$):

	$B=1$	$B=256$	params	ctrl pts
Catmull-Rom (free points)	779 μs	10365 μs	320	5
CR–Chebyshev ($g=12, k=5$)	700 μs	10588 μs	320	12

Same parameters, **2.4× the spline resolution**, smooth-by-construction (band-limited control points $\Rightarrow$ a regulariser), and the basis is cached. It is *not* a raw $B=1$ speed win (the CR gather still dominates) — but it has a sharper payoff: because the trained function *lies on a degree- k Chebyshev polynomial*, it is Chebyshev-representable, so the model can be **trained with CR locality and deployed with the cheap Chebyshev matmul** (the $3\times$ $B=1$ path), with a controlled interpolation error to validate. The hybrid is the *bridge* between the two bases: CR’s training quality, Chebyshev’s rollout speed.

4 Sparsity — a scaling lever, not a small-graph win

The signed adjacency $A_\pm$ and the star-incidence are sparse; the current aggregation is a *dense einsum* (`DenseBatchedBackend`), with a `SparseScatterBackend` (`torch.sparse.mm`) already present. For the current robots ($N \sim 6\text{--}14$ vertices) dense wins — sparse-op overhead exceeds the $O(N^2 B d)$ dense compute. Sparsity pays when N grows: **collaborative k -arm graphs**, large emitted robots, and the star-expanded combinatorial designs (many blocks). Two caveats: (i) B^L *densifies* with L (the L -hop reachability fills in), so a sparse SA-HSiKAN helps only for small L on sparse graphs; (ii) the CR gather is *already* local-support “sparse” (4 control points/sample). Recommendation: make the backend **N -adaptive** (auto-select dense vs. sparse at a profiled crossover), so the same code scales from the 2-arm grasp to the big collaborative graphs without a rewrite.

5 Recommendation

1. **Batch the rollout** (vectorised envs) — the highest-leverage, model-agnostic fix ($17\times$); re-frames the paper from “HSiKAN is launch-bound” to “fine-grained KAN message-passing

is dispatch-bound at $B=1$ and amortises $17\times$ under batched rollout.” Needs trainer/env vectorisation (real work).

2. **CR–Chebyshev hybrid** — ship as the default KAN cell: more resolution/parameter, smoother, and the train-CR / deploy-Chebyshev bridge for the rollout.
3. **SA-HSiKAN** (B^L) for the fixed-topology deploy path (done).
4. **N -adaptive sparse backend** for the collaborative/large-graph scaling.
5. `torch.compile`/Triton later, on the GPU box (free there; the $\sim 10\times$ GPU win).

These compose: batched rollout ($17\times$) $\times$ CR–Chebyshev deploy ($3\times$) $\times$ sparse at scale — without freezing the model or needing a compiler on Windows.